FPPD · FUNDAMENTOS DE PROCESSAMENTO PARALELO E DISTRIBUÍDO · TRABALHO 1

Concorrência e Sincronização: o Problema dos Filósofos Jantando

Grupo: Eduardo Pacheco Lobo · Igor Xavier Nunes · Rodrigo Fehlauer Lauermann
Código: github.com/eduardolobocode/fppd-t1-filosofos · Go 1.27 · outubro de 2026

Implementamos e medimos quatro versões do problema em Go. Todas usam os mesmos parâmetros (5 filósofos, 20 ciclos, pensar 20 ms e comer 30 ms, na proporção 2:3 do `filo_seq.go` de referência) e o mesmo esqueleto de ciclo: *pensar* → *pegar garfos* → *comer* → *soltar garfos*. Entre V1, V2 e V3 só mudam as funções `pegarGarfos` e `soltarGarfos`, então qualquer diferença medida vem da estratégia de sincronização.

1. Soluções e mecanismos de sincronização

VERSÃO	COMO FUNCIONA	MECANISMOS (GO)	DEADLOCK?
V0 sequencial	Um laço executa os filósofos um depois do outro, ciclo a ciclo. É a versão do professor com 20 ciclos e tempos em ms. Serve de linha de base (pior caso).	Nenhum: não há disputa por garfos.	Impossível.
V1 ingênua	Uma goroutine por filósofo; cada garfo é um mutex. Todos pegam o garfo esquerdo (`i`) e depois o direito (`(i+1)%5`). Um atraso de 10 ms entre os dois `Lock` torna o deadlock reproduzível na demonstração.	`go`, `sync.Mutex` por garfo, `sync.WaitGroup`.	Sim: as 4 condições de Coffman estão presentes.
V2 mesa global	A V1 (sem o atraso) com um mutex global, a "mesa", trancado do momento em que o filósofo vai pegar os garfos até soltá-los. Só um filósofo por vez pega, come e solta.	Mutex global de granularidade grossa + mutex por garfo.	Não: só um disputa garfos por vez, logo não há espera circular.
V3 ordenação	A V1 (sem o atraso) com uma regra: pegar sempre o garfo de **menor número** primeiro. Só o F4, que usa g4 e g0, muda de comportamento (pega g0 antes de g4).	Mutex por garfo (granularidade fina) + ordem global de aquisição.	Não: a ordem global impede que o ciclo de espera se feche.

2. Análise comparativa

3,2×
mais rápida: V3 em relação à V0 (1,62 s contra 5,12 s)

4,1×
mais espera pelos garfos na V2 do que na V3 (100 ms contra 25 ms)

10/10
execuções da V1 em deadlock com o atraso (17/50 sem ele)

Corretude. A V1 trava: na execução oficial os cinco filósofos pegaram o garfo esquerdo em 20,6 ms, ficaram esperando o direito, e o runtime do Go encerrou o programa com `fatal error: all goroutines are asleep - deadlock!`. V2 e V3 completaram os 20 ciclos de cada filósofo nas 90 execuções feitas (todos os cenários).

Desempenho. Entre as versões corretas, a V3 foi a mais rápida (1,62 s), 1,90× mais rápida que a V2 (3,08 s) e 3,16× mais rápida que a V0 (5,12 s). A V2 é correta, mas serializa as refeições: nunca houve mais de 1 filósofo comendo e a espera média foi de 99,7 ms. Mesmo assim ela supera a V0, porque *pensar* continua em paralelo: a sincronização excessiva só custa na parte que ela serializa. A V3 atingiu 2 filósofos comendo ao mesmo tempo, o máximo possível com 5 garfos.

Teoria × medição. Os tempos ficaram próximos dos mínimos teóricos (V0: 100 × 50 ms = 5 s; V2: 100 refeições × 30 ms em fila = 3 s; V3: duas refeições por vez = 1,5 s), com sobrecusto de 2,3%, 2,6% e 8,0%, respectivamente. A dispersão foi pequena (desvio padrão de 15 ms, 8 ms e 57 ms em 30 execuções).

Outros achados (seção 4):

- **A carga decide o custo da V2:** com pouca disputa ela leva só 1,05× o tempo da V3; com muita disputa, 1,93×.
- **A largada é o momento crítico:** sem o atraso, a V1 travou em 17 de 50 execuções, todas no primeiro ciclo; na V3, a largada forma uma corrente de espera e o primeiro par só come junto em 81,2 ms.
- **Concorrência não é paralelismo:** com um único núcleo (`GOMAXPROCS=1`) nenhuma versão ficou mais lenta, porque o gargalo são os garfos, não a CPU.

3. Resultados experimentais

Cenário base: pensar 20 ms, comer 30 ms, 20 ciclos. V0, V2 e V3: média de 30 execuções (6 rodadas do programa × 5 execuções); abaixo do valor, desvio padrão e faixa mín–máx. A V1 é analisada à parte porque não termina. Espera = tempo entre pedir e obter os dois garfos. Comendo ao mesmo tempo = contador protegido por um mutex usado só para medir. Concorrência média = 100 refeições × 30 ms ÷ tempo total.

Métrica	V0 · sequencial	V1 · deadlock	V2 · mesa global	V3 · ordenação
Tempo total	5,12 s ± 15 ms · 5,08 s–5,14 s	não termina (deadlock)	3,08 s ± 8 ms · 3,06 s–3,10 s	1,62 s ± 57 ms · 1,54 s–1,76 s
Mínimo teórico	5,0 s medido: +2,3%	—	3,0 s medido: +2,6%	1,5 s medido: +8,0%
Espera média pelos garfos	0,0 ms ± 0,0 ms	infinita	99,7 ms ± 0,3 ms	24,5 ms ± 0,5 ms
Maior espera observada	0,5 ms	infinita	125,0 ms	151,5 ms
Máx. comendo ao mesmo tempo	1	0	1	2
Concorrência média (comendo)	0,59	0	0,97	1,85
Ciclos por filósofo	20 em todas	0 (trava no 1º ciclo)	20 em todas	20 em todas

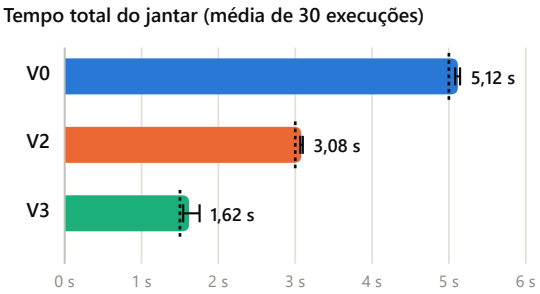


Figura 1. Barra: média; traço preto: faixa mín–máx; tracejado: mínimo teórico.

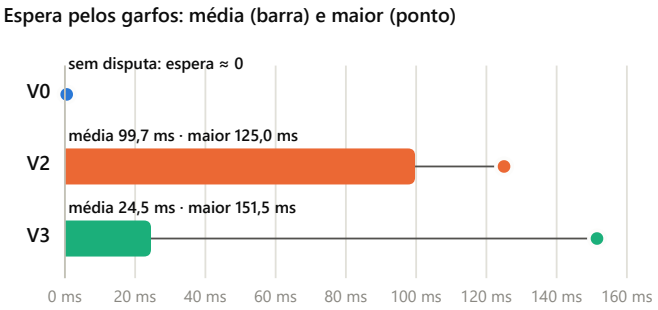


Figura 2. A V2 tem a maior espera média; a maior espera isolada foi na V3, na largada do jantar (seção 4.8). Na V0 ninguém espera.

Windows PowerShell

```
PS fppd-t1-filosofos> go run ./v1-deadlock
V1 - Versão concorrente sujeita a deadlock
Go go1.27.0 | windows/amd64 | 20 núcleos lógicos | GOMAXPROCS=20
5 filósofos | 20 ciclos | pensar 20ms | comer 30ms | atraso entre garfo
s 10ms
     0.0 ms  F4: P1
     0.0 ms  F0: P1
     0.0 ms  F1: P1
     0.0 ms  F2: P1
     0.0 ms  F3: P1
    20.6 ms  F0: pegou g0 (esquerdo)
    20.6 ms  F3: pegou g3 (esquerdo)
    20.6 ms  F2: pegou g2 (esquerdo)
    20.6 ms  F1: pegou g1 (esquerdo)
    20.6 ms  F4: pegou g4 (esquerdo)
    31.1 ms  F0: tentando pegar g1 (direito)...
    31.1 ms  F2: tentando pegar g3 (direito)...
    31.1 ms  F3: tentando pegar g4 (direito)...
    31.1 ms  F4: tentando pegar g0 (direito)...
    31.1 ms  F1: tentando pegar g2 (direito)...
fatal error: all goroutines are asleep - deadlock!
goroutine 1 [sync.WaitGroup.Wait]:
sync.runtime_SemacquireWaitGroup(0x27fab29c8b70?, 0x0?)
...
goroutine 6 [sync.Mutex.Lock]:
main.pegarGarfos(0x1312d00?)
    .../v1-deadlock/main.go:55 +0x18d
...
```

Figura 3. Evidência do deadlock da V1 (saída real, trecho): cada filósofo pega o garfo esquerdo e fica tentando pegar o direito; o runtime detecta que todas as goroutines estão bloqueadas. As outras quatro goroutines de filósofo aparecem iguais, bloqueadas em `sync.Mutex.Lock` na linha 55.

Windows PowerShell

```
PS fppd-t1-filosofos> go run ./v3-ordenacao
V3 - Versão concorrente aprimorada (ordenação de recursos)
Go go1.27.0 | windows/amd64 | 20 núcleos lógicos | GOMAXPROCS=20
5 filósofos | 20 ciclos | pensar 20ms | comer 30ms | 5 execuções
Linha do tempo da execução 1 (P = pensando, C = comendo, número = cicl
o)
   instante  F0      F1      F2      F3      F4
     0.0 ms                                  P1
     0.0 ms  P1
     0.0 ms          P1
     0.0 ms                          P1
     0.0 ms                  P1
    20.6 ms                          C1
    50.9 ms                          P2
    50.9 ms                  C1
    81.2 ms                  P2
    81.2 ms                          C2
    81.2 ms          C1
   111.5 ms          P2
   111.5 ms                          P3
   111.5 ms  C1
   111.5 ms                  C2
   141.8 ms                  P3
   141.8 ms  P2
   141.8 ms                          C3
   141.8 ms          C2
   172.1 ms          P3
   172.1 ms                          P4
...
```

Figura 4. Linha do tempo da V3 (saída real, trecho). Cada coluna é um filósofo; P = pensando, C = comendo. Na largada só o F3 come (os outros formam uma corrente de espera); a partir de 81,2 ms duas refeições começam no mesmo instante.

4. Análise dos resultados

As respostas abaixo seguem as 12 questões propostas no enunciado (4.1 a 4.12) e acrescentam quatro análises que os dados permitiram (4.13 a 4.16). Os números vêm do cenário base, salvo indicação.

4.1 Como funciona cada solução?

As quatro versões compartilham a função `umCiclo` (Apêndice A): o filósofo pensa (`time.Sleep` de 20 ms), mede o tempo que leva para obter os garfos, come (30 ms), solta os garfos e conta o ciclo.

- **V0:** dois laços aninhados, ciclo de 1 a 20 e filósofo de 0 a 4. Como nada roda em paralelo, os garfos não precisam de proteção e `pegarGarfos` é vazia. O tempo é a soma de tudo: 20 × 5 × 50 ms = 5 s.
- **V1:** o `main` cria uma goroutine por filósofo e espera todas com um `sync.WaitGroup`. `pegarGarfos` faz `Lock` no garfo esquerdo e depois no direito.
- **V2:** igual à V1, mas `pegarGarfos` começa com `mesa.Lock()` e `soltarGarfos` termina com `mesa.Unlock()`. Do pedido dos garfos até a devolução, o filósofo tem a mesa inteira só para ele.
- **V3:** igual à V1, mas antes dos `Lock` os dois índices são ordenados. F0 a F3 já pegavam o menor primeiro; o F4, cujos garfos são g4 e g0, passa a pegar g0 primeiro. Essa assimetria é toda a diferença.

4.2 Quais mecanismos de sincronização são utilizados?

- **Exclusão mútua por mutex** (V1, V2, V3): cada garfo é um recurso não compartilhável protegido por um lock.
- **Lock global de granularidade grossa** (V2): um único mutex protege a mesa inteira, transformando a seção de pegar, comer e soltar em uma seção crítica única.
- **Ordenação de recursos** (V3): não é uma primitiva, é um protocolo de uso dos mutexes. Todos adquirem os locks na mesma ordem global.
- **Espera pelo término** (V1, V2, V3): `sync.WaitGroup` faz o `main` aguardar os cinco filósofos.
- **Medição sem interferência:** as esperas e ciclos de cada filósofo ficam em posições de vetor só dele (sem lock); só o contador de "comendo agora" é compartilhado e usa um mutex próprio, fora da estratégia avaliada.

4.3 Como esses mecanismos são implementados em Go?

- **Goroutines:** tarefas leves criadas com `go f()`. O runtime as distribui entre threads do sistema (modelo M:N); nesta máquina, `GOMAXPROCS` = 20. Uma goroutine em `time.Sleep` ou bloqueada num mutex é estacionada e não consome CPU.
- **`sync.Mutex`:** `Lock()` bloqueia se o mutex estiver ocupado; `Unlock()` libera e acorda um interessado. O valor zero já é um mutex livre. Ele *não* é reentrante e não tem dono: nenhuma goroutine consegue tomá-lo de outra (é a "ausência de preempção"). Por padrão não é FIFO, mas entra em *modo starvation* quando alguém espera mais de 1 ms e passa a atender por ordem de chegada.
- **`sync.WaitGroup`:** contador com `Add`, `Done` e `Wait`.
- **Detector de deadlock do runtime:** se todas as goroutines estão bloqueadas, o programa é abortado com a pilha de cada uma (Figura 3). Ele só funciona quando *todas* estão paradas.
- Go não tem semáforo nem monitor nativos (seriam feitos com canal com buffer e com `sync.Cond`); nenhuma das estratégias escolhidas precisou deles.

v1-deadlock/main.go · pegarGarfos

```go
func pegarGarfos(id int) {
    esquerdo, direito := id, (id+1)%numFilosofos

    garfos[esquerdo].Lock()
    imprimir(true, id, fmt.Sprintf("pegou g%d (esquerdo)", esquerdo))

    time.Sleep(atrasoEntreGarfos) // alarga a janela perigosa (ver comentário no topo)

    imprimir(true, id, fmt.Sprintf("tentando pegar g%d (direito)...", direito))
    garfos[direito].Lock() // se o vizinho segura este garfo e espera o dele, ninguém sai daqui
    imprimir(true, id, fmt.Sprintf("pegou g%d (direito)", direito))
}
```

4.4 Por que ocorre deadlock na V1?

Porque as quatro condições de Coffman valem ao mesmo tempo, e a ordem "esquerdo, depois direito", igual para todos, fecha o círculo. A saída oficial (Figura 3) mostra o estado final:

Filósofo	Segura	Espera	Quem tem o garfo esperado
F0	g0	g1	F1
F1	g1	g2	F2
F2	g2	g3	F3
F3	g3	g4	F4
F4	g4	g0	F0 (fecha o ciclo)

1. **Exclusão mútua:** cada garfo é um `sync.Mutex`; só um filósofo o segura.
2. **Posse e espera:** cada filósofo segura o garfo esquerdo enquanto espera o direito.
3. **Ausência de preempção:** nenhuma goroutine pode tirar o mutex de outra.
4. **Espera circular:** F0 espera F1, que espera F2, … e F4 espera F0.

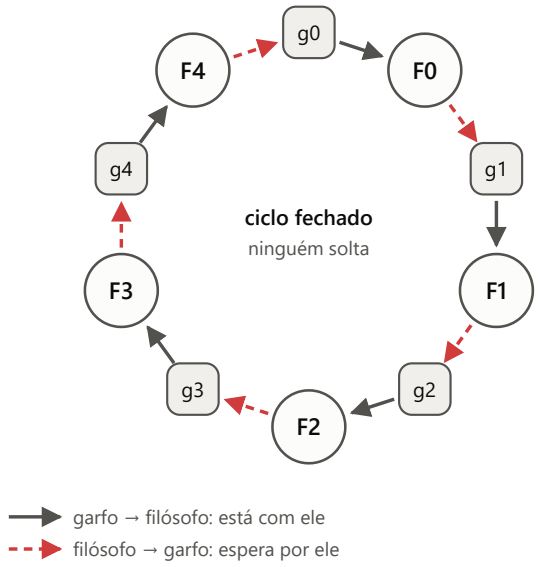


Figura 5. Grafo de espera da execução oficial: o ciclo fechado é o deadlock.

O atraso de 10 ms entre os dois `Lock` não cria o defeito: ele apenas alarga a janela em que todos seguram o garfo esquerdo. Sem o atraso, o mesmo código travou em **17 de 50** execuções (seção 4.13): o defeito existe sempre, mas só se manifesta quando o escalonador produz a intercalação ruim.

4.5 Como V2 e V3 evitam deadlock?

As duas quebram a **espera circular**, por caminhos diferentes.

- **V2:** só o dono da mesa pode segurar ou esperar garfos. Um ciclo de espera precisa de pelo menos dois filósofos segurando recursos, e isso nunca acontece. Na prática, os mutexes dos garfos ficam até redundantes.
- **V3:** suponha um ciclo de espera. Cada filósofo nele segura um garfo e espera outro de número *maior* (é a regra). Percorrendo o ciclo, os números teriam de crescer a cada passo e voltar ao início, o que é impossível. Logo não há ciclo nem deadlock, e sem limitar quem não disputa garfos.

Evidência: as 90 execuções de V2 e V3 (três cenários de carga e com um único núcleo) terminaram com 20 ciclos para cada filósofo.

v2-mesa/main.go · mesa global

```
var garfos [numFilosofos]sync.Mutex
var mesa sync.Mutex

func pegarGarfos(id int) {
    esquerdo, direito := id, (id+1)%numFilosofos
    mesa.Lock() // só um filósofo por vez a partir
daqui (fica trancada até soltarGarfos)
    garfos[esquerdo].Lock()
    garfos[direito].Lock()
}

func soltarGarfos(id int) {
    esquerdo, direito := id, (id+1)%numFilosofos
    garfos[direito].Unlock()
    garfos[esquerdo].Unlock()
    mesa.Unlock()
}
```

v3-ordenacao/main.go · pegarGarfos

```
func pegarGarfos(id int) {
    primeiro, segundo := id, (id+1)%numFilosofos
    if primeiro > segundo {
        primeiro, segundo = segundo, primeiro // só o
F4 inverte: g0 antes de g4
    }
    garfos[primeiro].Lock()
    garfos[segundo].Lock()
}
```

4.6 Existe possibilidade de starvation? Justifique.

Na V0, não: a ordem é fixa. **Na V2 e na V3, sim em teoria**, porque nenhuma das duas garante a vez de cada filósofo: um mutex não é uma fila, e na V3 dois vizinhos podem se alternar e deixar o do meio sempre encontrando um garfo ocupado. **Na prática, não observamos starvation:** todos completaram os 20 ciclos e a espera foi parecida entre os filósofos (Figura 6). Na V2, as médias por filósofo ficaram entre 98,5 ms e 100,4 ms; o modo starvation do mutex do Go aproxima a mesa de uma fila. Na V3, ficaram entre 20,6 ms (F3) e 29,5 ms (F4), e a maior espera de um filósofo foi 151,5 ms, cerca de 5,0 refeições. F0 e F4 esperam um pouco mais porque os dois começam pelo mesmo garfo, g0, efeito direto da regra de ordenação. A maior espera isolada da V3 não é starvation em regime: é a espera da largada do jantar, explicada em 4.8 e 4.9.

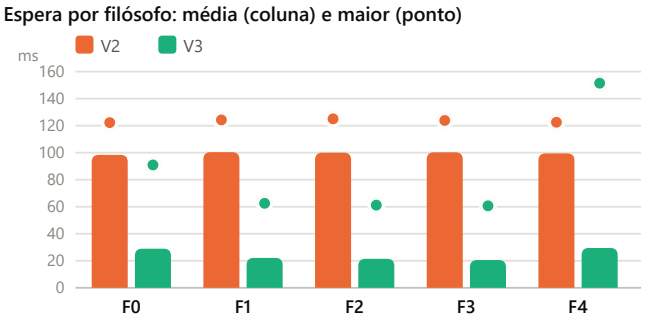


Figura 6. Espera por filósofo (média de 6 rodadas; ponto = maior espera). Nenhum filósofo foi sistematicamente preterido.

Com só 20 ciclos e tempos fixos, a carga é regular e favorece a justiça. Uma garantia formal exigiria um mecanismo com ordem de chegada explícita, como um "garçom" que atende por fila de senhas.

Na V2, as colunas são praticamente iguais: a mesa funciona como uma fila. Na V3, F1, F2 e F3 esperam menos que F0 e F4, os dois que disputam o g0 como primeiro garfo. O ponto alto do F4 é a espera da largada.

4.7 Qual solução apresentou maior tempo total de execução?

Desconsiderando a V1, que nunca termina, a **V0**: 5,12 s em média, contra 5 s teóricos. Era o esperado, porque ela soma todos os tempos de pensar e comer. Entre as concorrentes, a **V2** foi a mais lenta (3,08 s), 1,90× o tempo da V3.

4.8 Qual solução apresentou maior tempo de espera pelos recursos?

Na média, a V2: 99,7 ms, contra 24,5 ms na V3 (V0: zero; V1: infinita). O número da V2 bate com um cálculo simples: como a mesa atende um por vez, em regime cada filósofo volta à mesa a cada 5 × 30 = 150 ms; descontando 30 ms comendo e 20 ms pensando, sobram cerca de 100 ms de espera. A maior espera da V2 (125,0 ms) corresponde a aguardar os outros quatro comerem: 4 × 30 = 120 ms.

Na maior espera isolada, a V3: 151,5 ms. Ela vem da largada: na execução oficial, o F4 só comeu pela primeira vez 151,5 ms depois de ficar com fome, preso na corrente de espera descrita em 4.9. Passada a largada, as esperas da V3 caem para a faixa de 20 a 30 ms. Em resumo: a V2 faz todos esperarem muito o tempo todo; a V3 faz poucos esperarem bastante, e só no começo.

4.9 Quantos filósofos conseguiram comer simultaneamente em cada solução?

V0: 1 · V1: 0 (trava antes da primeira refeição) · V2: 1 · V3: 2. Dois é o limite teórico: cada refeição ocupa 2 dos 5 garfos e dois vizinhos nunca comem juntos. A Figura 7 mostra a diferença no tempo: na V2 as refeições formam uma fila contínua, sem sobreposição; na V3, passada a largada, aparecem quase sempre dois filósofos não vizinhos comendo em paralelo.

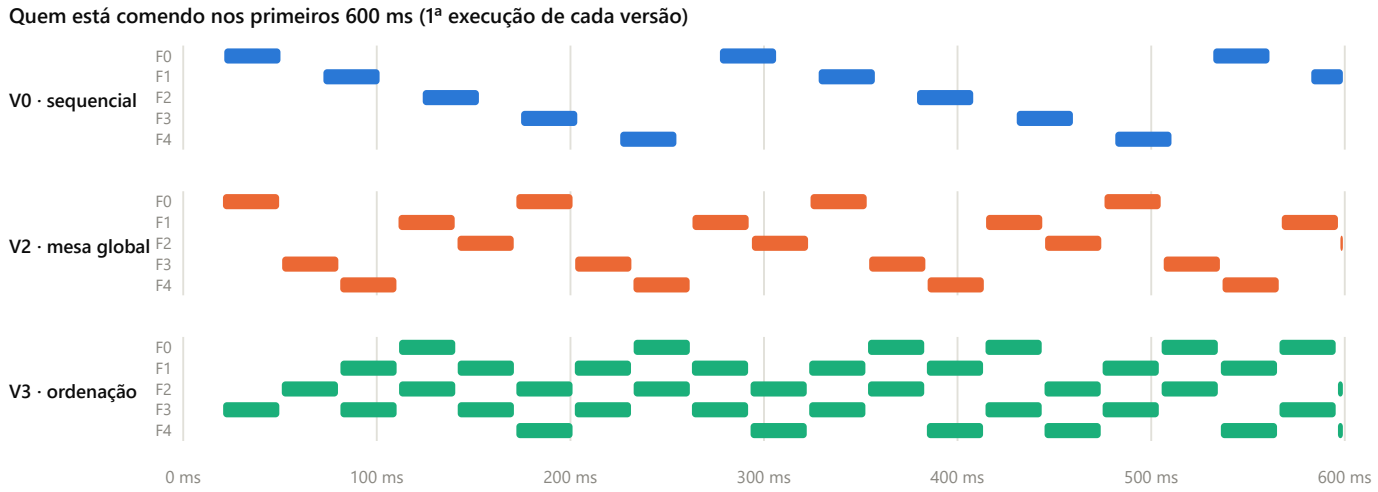
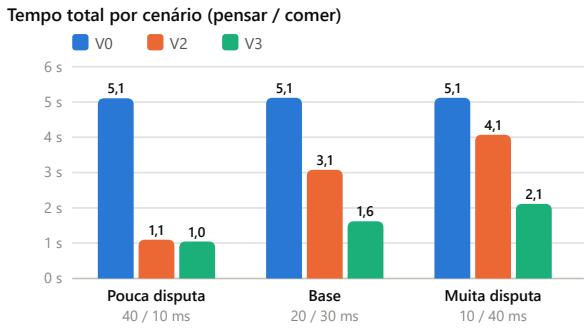


Figura 7. Refeições ao longo do tempo, reconstruídas da linha do tempo impressa pelos programas. A V0 percorre um filósofo por vez com pausas para pensar; a V2 elimina as pausas mas não a fila; a V3 forma pares depois de uma largada em fila.

Um efeito que só a linha do tempo mostra. Como todos começam a pensar no mesmo instante, todos ficam com fome juntos. Na V3, F0 pega g0, F1 pega g1, F2 pega g2 e F3 pega g3; o F4 tenta g0 (seu primeiro garfo pela regra) e fica bloqueado sem segurar nada. Assim o F3 consegue g4 e come sozinho, enquanto F2, F1 e F0 formam uma **corrente de espera**, cada um segurando o garfo de que o vizinho precisa. A corrente se desfaz um elo por vez, e só em 81,2 ms aparece o primeiro par comendo junto. Ou seja: a ordenação elimina o *ciclo*, mas não a *posse e espera*, que ainda pode serializar filósofos por um tempo. É por isso que a V3 fica um pouco acima do 1,5 s teórico e varia mais entre execuções.

4.10 Qual foi o impacto da sincronização excessiva da V2?

No cenário base, a mesa global deixou o programa 90% mais lento que a V3, multiplicou a espera média por 4,1 e reduziu a concorrência máxima de 2 para 1. O impacto, porém, depende da carga. Rodamos mais dois cenários mantendo pensar + comer = 50 ms (por isso a V0 leva sempre cerca de 5 s):



Cenário	V0	V2	V3	V2÷V3	Espera V2	Espera V3
Pouca disputa 40 / 10 ms	5,10 s	1,10 s	1,05 s	1,05×	2,7 ms	0,4 ms
Base 20 / 30 ms	5,12 s	3,08 s	1,62 s	1,90×	99,7 ms	24,5 ms
Muita disputa 10 / 40 ms	5,12 s	4,07 s	2,11 s	1,93×	148,2 ms	48,9 ms

Com **pouca disputa** (comer é curto), a mesa quase nunca está ocupada e a V2 fica perto da V3 (1,05×). Com **muita disputa**, a fila da mesa domina: a V2 leva 4,07 s, mais perto da V0 (5,12 s) do que da V3 (2,11 s).

Figura 8. Tempo total médio (5 execuções por cenário; base: 30).

Conclusão: o custo da sincronização excessiva é proporcional ao quanto a seção serializada pesa no trabalho total. Ele é baixo quando a seção crítica é curta e cresce até anular o paralelismo quando ela domina.

4.11 A solução com maior sincronização foi necessariamente a de melhor desempenho?

Não, foi o contrário. V2 e V3 são igualmente corretas (nenhuma trava), mas a V2 sincroniza mais: tranca três mutexes por refeição em vez de dois, e um deles bloqueia todos os filósofos, inclusive os que não disputam garfo nenhum. O resultado foi 1,90× mais tempo e 4,1× mais espera. Sincronização garante corretude, mas cada lock a mais é uma fila a mais. A melhor solução sincroniza só o necessário: protege cada garfo (granularidade fina) e evita o deadlock com uma regra de ordem, sem bloquear quem não compete.

4.12 Qual é a relação entre o grau de concorrência observado e o tempo total de execução?

Inversa. O trabalho de comer é fixo: 100 refeições de 30 ms, ou 3.000 ms. O que muda entre as versões é quantas refeições acontecem ao mesmo tempo. A **concorrência média** (3.000 ms divididos pelo tempo do jantar) é o número médio de filósofos comendo a cada instante; a **ocupação** compara esse número com o máximo que a versão permite.

Versão	Máx. comendo junto	Concorrência média	Ocupação dos lugares	Tempo total	Ganho sobre a V0
V0	1	0,59	59%	5,12 s	1,00×
V2	1	0,97	97%	3,08 s	1,66×
V3	2	1,85	93%	1,62 s	3,16×

Duas leituras saem da tabela. Primeiro, V2 e V3 mantêm seus "lugares" à mesa quase sempre ocupados (97% e 93%): a diferença de tempo entre elas vem inteiramente de quantos lugares existem, 1 ou 2. Dobrar a concorrência permitida reduziu o tempo quase à metade. Segundo, a V0 tem concorrência abaixo de 1 porque, enquanto um filósofo pensa, ninguém come. O ganho tem dois tetos: o físico (no máximo 2 comendo ao mesmo tempo) e o de cada filósofo, que precisa de pelo menos 20 × (20 + 30) ms = 1 s para os próprios ciclos. No cenário de pouca disputa é esse segundo teto que manda, e por isso V2 e V3 se aproximam (Figura 8).

4.13 O deadlock da V1 acontece sem o atraso artificial?

Sim, em parte das execuções. Sem o atraso, a V1 travou em **17 de 50** execuções (34%); com o atraso, em 10 de 10. A frequência também oscila entre baterias: num teste exploratório anterior, a mesma versão sem atraso travou em apenas 2 de 20 execuções. Em todas as execuções que travaram, o deadlock aconteceu no primeiro ciclo, antes de qualquer refeição. É o momento de maior risco: como todos começam a pensar juntos e pensam pelo mesmo tempo, todos ficam com fome no mesmo instante e pegam o garfo esquerdo quase simultaneamente. Depois da largada, as refeições se desencontram e a intercalação fatal fica improvável.

É o comportamento típico de defeitos de concorrência: o programa passa em vários testes e falha de repente, conforme a ordem escolhida pelo escalonador. Por isso a análise das quatro condições é mais confiável que "testar e ver se trava".

4.14 Mais núcleos deixam o jantar mais rápido?

Versão	GOMAXPROCS = 20 (padrão)	GOMAXPROCS = 1	Diferença	Máx. comendo (1 núcleo)
V0	5,12 s	5,11 s	−0,2%	1
V2	3,08 s	3,07 s	−0,4%	1
V3	1,62 s	1,55 s	−4,6%	2

Não. Com `GOMAXPROCS=1`, um único núcleo executa todas as goroutines e nenhuma versão ficou mais lenta; a V3 manteve 2 filósofos comendo ao mesmo tempo. Isso acontece porque pensar e comer são `Sleep`: as goroutines passam quase todo o tempo bloqueadas, sem usar CPU. O gargalo é a disputa pelos garfos, não o processador. É a diferença entre concorrência (estruturar o programa em tarefas independentes) e paralelismo (executá-las ao mesmo tempo em núcleos distintos).

A V3 com um núcleo foi até um pouco mais rápida e bem mais estável (desvio padrão de 3,9 ms contra 57,2 ms). Uma explicação provável é que, com um só núcleo, as goroutines acordam numa ordem mais regular e a corrente de espera da largada se desfaz sempre do mesmo jeito. Como esse teste teve 5 execuções por versão, a diferença deve ser lida como indicativa.

4.15 As medições são confiáveis?

- **Repetição:** 30 execuções por versão no cenário base (o enunciado pede 5), com desvio padrão de 15 ms (V0), 8 ms (V2) e 57 ms (V3). A V3 varia mais porque o pareamento de quem come junto muda a cada execução.
- **Sobrecusto previsível:** a V0 levou 117 ms a mais que os 5 s teóricos, cerca de 0,59 ms por `Sleep` (são 200), o custo de acordar uma goroutine no Windows.
- **Sem condições de corrida:** V0, V2 e V3 rodaram com o detector de corrida do Go (`go run -race`) sem nenhum alerta. Com `-race`, a V1 não aborta: o detector desativa a detecção automática de deadlock e o programa fica parado em silêncio, como aconteceria num sistema real.
- **Impressão:** a linha do tempo só é impressa na 1ª execução de cada rodada. Na V0 e na V2 a diferença entre a 1ª execução e as outras foi desprezível (+3,4 ms e −7,2 ms). Na V3, a 1ª execução foi em média 61,4 ms mais lenta, da ordem do próprio desvio padrão da versão (57,2 ms); como só 6 das 30 execuções imprimem, o efeito na média da V3 é de cerca de 12 ms.
- **Limitações:** uma única máquina e um único sistema operacional; tempos de pensar e comer fixos (sem aleatoriedade), o que torna o comportamento mais regular do que num sistema real.

4.16 Conclusão

Os dados confirmam a teoria em todos os pontos. A versão ingênua (V1) é concorrente mas incorreta: as quatro condições de Coffman produzem deadlock, às vezes raro, mas sempre possível. Resolver o deadlock com um lock global (V2) devolve a corretude ao preço da concorrência: o jantar vira uma fila, a espera é a maior de todas e o desempenho fica entre a versão sequencial e a ideal, mais perto da sequencial quanto maior a disputa. Resolver o mesmo problema quebrando só a espera circular (V3) mantém a granularidade fina, atinge o máximo teórico de dois filósofos comendo ao mesmo tempo e entrega o menor tempo (1,62 s, 3,2× mais rápido que a V0). A lição central é que a quantidade de sincronização não mede a qualidade de uma solução: o melhor desempenho veio de sincronizar menos, mas no lugar certo.

5. Ambiente de execução e reprodução

Linguagem	Go 1.27.0 (windows/amd64); módulo exige Go ≥ 1.22
Sistema operacional	Windows 11 Home Single Language
Processador	13th Gen Intel Core i7-13700H
Núcleos	14 núcleos físicos, 20 lógicos (`runtime.NumCPU()` = 20, `GOMAXPROCS` = 20)
Execução	`go run ./v0-sequencial` · `go run ./v1-deadlock` · `go run ./v2-mesa` · `go run ./v3-ordenacao` (na raiz do repositório)
Compilação	`go build -o v3.exe ./v3-ordenacao` (idem para as outras); verificação: `go vet ./...`, `gofmt -l .`, `go run -race`
Experimentos	`py relatorio/experimentos.py` (gera `resultados/*.csv`) e `py relatorio/gerar_relatorio.py` (gera este PDF)
Repositório	https://github.com/eduardolobocode/fppd-t1-filosofos

Apêndice A · Código-fonte (trechos)

Código completo e formatado com `gofmt` no repositório. Os trechos abaixo são extraídos automaticamente dos arquivos `main.go`.

umCiclo · igual nas quatro versões

```
func umCiclo(id, ciclo int, m *metricas, mostrar bool) {
    imprimir(mostrar, id, fmt.Sprintf("P%d", ciclo))
    time.Sleep(tempoPensar)

    inicioEspera := time.Now()
    pegarGarfos(id)
    m.registrarEspera(id, time.Since(inicioEspera))

    m.comecouComer()
    imprimir(mostrar, id, fmt.Sprintf("C%d", ciclo))
    time.Sleep(tempoComer)
    m.terminouComer() // antes de soltar os garfos, para o vizinho não começar antes da contagem baixar

    soltarGarfos(id)
    m.ciclos[id]++
}
```

v0-sequencial/main.go · jantar (laço sequencial)

```
func jantar(mostrar bool) resultado {
    m := &metricas{}
    inicio := time.Now()
    inicioJantar = inicio

    for ciclo := 1; ciclo <= numCiclos; ciclo++ {
        for id := 0; id < numFilosofos; id++ {
            umCiclo(id, ciclo, m, mostrar)
        }
    }

    return m.resumo(time.Since(inicio))
}
```

jantar · uma goroutine por filósofo (V1, V2, V3)

```
func jantar(mostrar bool) resultado {
    m := &metricas{}
    var wg sync.WaitGroup
    inicio := time.Now()
    inicioJantar = inicio

    for id := 0; id < numFilosofos; id++ {
        wg.Add(1)
        go filosofo(id, m, mostrar, &wg)
    }
    wg.Wait()

    return m.resumo(time.Since(inicio))
}
```

métricas · contagem de quem está comendo

```
func (m *metricas) comecouComer() {
    m.mu.Lock()
    m.comendo++
    if m.comendo > m.maxComendo {
        m.maxComendo = m.comendo
    }
    m.mu.Unlock()
}

func (m *metricas) terminouComer() {
    m.mu.Lock()
    m.comendo--
    m.mu.Unlock()
}
```